

机器学习 实验报告

实验一：基于 kNN 的手写数字识别

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

实验数据集：Semeion 手写数字数据集

完成日期：2026 年 10 月 9 日

GitHub 仓库

https://github.com/ZR-1N/Machine_Learning_NKU

南开大学

目录

1	实验概述	1
1.1	实验目的	1
1.2	实验内容与进阶要求	1
2	数据集与实验环境	1
2.1	数据集说明	1
2.2	数据文件关系验证	2
2.3	实验环境	2
3	kNN 算法原理	2
3.1	基本思想	2
3.2	距离度量	3
3.3	k 值选择	3
4	自写 kNN 与 LOO 实现	3
4.1	数据读取与标签转换	3
4.2	向量化距离计算	3
4.3	多数表决与平局规则	4
4.4	留一法交叉验证	4
5	样本可视化与数据检查	5
5.1	导论示例代码运行情况	5
6	Weka 对比实验	5
6.1	ARFF 转换	5
6.2	Weka 参数设置	6
7	实验结果	6
7.1	自写程序结果	6
7.2	Weka 结果	7
7.3	自写程序与 Weka 对比	8
8	实验结果验证与分析	9
8.1	文档预期值与实际结果不一致	9
8.2	按类别表现分析	9

9 踩坑记录与解决方法	9
9.1 数据准备阶段	9
9.2 代码实现阶段	10
9.3 结果验证阶段	10
9.4 Weka 对比阶段	10
10 结论	10
参考资料	11
A 附录：本地运行输出	11

1 实验概述

1.1 实验目的

本实验围绕 Semeion 手写数字数据集实现 k 近邻分类器，并使用留一法交叉验证评估模型效果。结合课程导论文档与 kNN 课件，本实验的目标包括：

1. 理解 kNN 作为监督学习算法的基本思想，掌握距离度量、近邻数量 k 、投票规则等关键因素对分类结果的影响。
2. 使用 Python 与 NumPy 自行实现欧氏距离、多数表决和留一法交叉验证，不依赖现成分类器完成核心实验。
3. 使用 Weka 的 IBk 分类器复现实验，并比较自写程序与 Weka 的分类精度。
4. 分析实验中出现的格式、评估协议、默认参数和平局处理等问题，形成可复现、可解释的实验结论。

1.2 实验内容与进阶要求

基础任务要求实现自写 kNN 和 LOO 评估，并对 $k = 1, 3, 5$ 进行实验。中级要求使用 Weka 进行对照实验。进阶要求不仅记录运行结果，还需要说明思考过程、具体实现步骤和踩坑记录。因此，本报告在常规的算法原理、实现方法、实验结果之外，专门加入数据准备验证、Weka 参数对齐、预期值异常排查和误差来源分析。

2 数据集与实验环境

2.1 数据集说明

实验使用 Semeion Handwritten Digit 数据集。原始文件为 `semeion.data`，共 1593 行、266 列。每行前 256 列表示 16×16 二值手写数字图像展开后的像素，后 10 列表示 one-hot 编码标签。设原始行向量为

$$\mathbf{r} = (x_1, \dots, x_{256}, y_0, \dots, y_9),$$

其中 $x_i \in \{0, 1\}$ ，标签由

$$c = \arg \max_{j \in \{0, \dots, 9\}} y_j$$

得到。

本地检查结果显示，数据矩阵形状为 (1593, 266)，类别分布如表 1 所示。各类别样本数量接近，适合用于比较不同 k 值下的整体分类效果。

表 1: Semeion 数据集类别分布

类别	0	1	2	3	4	5	6	7	8	9
样本数	161	162	159	159	161	159	161	158	155	158

2.2 数据文件关系验证

实验文件夹中同时存在完整数据 `semeion.data` 以及两份划分文件 `semeion_train.txt`、`semeion_test.txt`。起初容易误以为训练集和测试集划分文件也应参与本实验。实际核对后可知，训练文件有 1293 行，测试文件有 300 行，二者合计 1593 行；用 NumPy 将二者拼接后按行排序，与 `semeion.data` 的行集合完全一致。因此，`train/test` 文件只是全集的一个划分版本。

由于导论文档第 5 节要求使用留一法，LOO 的定义是每次从完整数据集中留出一个样本测试，其余样本训练，所以本实验只使用 `semeion.data`。这一点直接影响实验协议：如果改用固定 `train/test` 划分，样本数量、评估方式和结果都不能与 LOO 对齐。

2.3 实验环境

表 2: 实验环境与工具

项目	配置或用途
操作系统	Windows 环境下运行 Python 与 Weka
Python 依赖	NumPy 用于矩阵运算，Matplotlib 用于图像展示
核心程序	<code>exp1.py</code> 实现 kNN 与 LOO， <code>knn.py</code> 展示样本图像
Weka 工具	Weka Explorer 中的 <code>lazy.IBk</code> 分类器
数据转换	<code>convert_to_arff.py</code> 将空格分隔数据转换为 Weka 原生 ARFF 格式

3 kNN 算法原理

3.1 基本思想

kNN 是一种直观的监督学习方法，可用于分类和回归。它没有显式的参数训练过程，而是在预测阶段计算待分类样本与训练集中所有样本的距离，选出距离最近的 k 个样本，再根据这些近邻的标签进行预测。课件中用“物以类聚，人以群分”概括了这一思想：如果一个样本在特征空间中接近某一类样本，它大概率也属于该类。

对于分类任务，kNN 的三个核心步骤为：

1. 数据准备：读入带标签数据，并将 one-hot 标签转换为整数类别。
2. 寻找近邻：计算测试样本与所有训练样本的距离，按距离升序排列。

3. 投票预测：选取前 k 个近邻，以多数表决得到预测类别。

3.2 距离度量

本实验使用欧氏距离。设样本 $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^{256}$ ，则二者距离为

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^{256} (x_{im} - x_{jm})^2}.$$

Semeion 的像素为 0/1 二值特征，因此欧氏距离与汉明距离存在单调关系：平方欧氏距离实际上等于两个二值向量不同位置的数量。使用欧氏距离可以保持与导论代码、Weka 默认距离设置一致。

3.3 k 值选择

k 是 kNN 的关键超参数。 k 过小时，模型容易受噪声或异常样本影响，分类边界更不稳定； k 过大时，模型会过度平滑局部结构，可能忽略细粒度差异。课件建议常用 $k = 3, 5, 7$ ，本实验按照导论文档要求比较 $k = 1, 3, 5$ 。

4 自写 kNN 与 LOO 实现

4.1 数据读取与标签转换

核心读取代码如下。前 256 列作为特征矩阵 X ，后 10 列通过 `argmax` 转换为 0 到 9 的整数标签。

Listing 1: 数据读取与标签转换

```
1 raw = np.loadtxt('data/semeion.data')
2 X = raw[:, :256]
3 y = np.argmax(raw[:, 256:], axis=1)
```

4.2 向量化距离计算

LOO 需要进行 1593 轮评估。每一轮中，一个样本作为测试样本，其余 1592 个样本作为训练样本。若使用纯 Python 双重循环，需要大约

$$1593 \times 1592 \times 256 \approx 6.5 \times 10^8$$

次逐维计算，运行时间会明显增加。因此，实验中使用 NumPy 广播一次性计算测试样本到所有训练样本的距离：

Listing 2: 欧氏距离的向量化实现

```
1 def euclidean_distances(X_train, x_test):
2     squares = (X_train - x_test) ** 2
3     dists = np.sqrt(squares.sum(axis=1))
4     return dists
```

这里 $X_{\text{train}} - x_{\text{test}}$ 会自动将一维测试向量按行广播到训练矩阵上，从而得到所有训练样本与测试样本的逐维差值。该实现简洁且高效，是本实验完成 LOO 的关键。

4.3 多数表决与平局规则

预测阶段先对距离排序，取前 k 个近邻标签，再使用 `np.bincount().argmax()` 得到票数最多的类别：

Listing 3: kNN 预测函数

```
1 def knn_predict(X_train, y_train, x_test, k):
2     dists = euclidean_distances(X_train, x_test)
3     k_idx = np.argsort(dists)[:k]
4     k_labels = y_train[k_idx]
5     pred = np.bincount(k_labels).argmax()
6     return pred
```

需要注意的是，`argmax()` 在多个类别票数相同时返回编号最小的类别。这是一个确定性的平局规则，但不一定与 Weka 完全相同。对于二值像素数据，距离并列很常见，平局规则会在 $k = 3$ 等情况下造成小幅结果差异。

4.4 留一法交叉验证

LOO 的实现如下。每轮删除第 i 个样本作为测试样本，其余样本用于预测，最后统计正确率。

Listing 4: LOO 评估函数

```
1 def loo_eval(X, y, k):
2     n = X.shape[0]
3     correct = 0
4     for i in range(n):
5         x_test = X[i]
6         X_train = np.delete(X, i, axis=0)
7         y_train = np.delete(y, i)
8         pred = knn_predict(X_train, y_train, x_test, k)
9         if pred == y[i]:
10             correct += 1
11     return correct / n
```

5 样本可视化与数据检查

5.1 导论示例代码运行情况

导论文档给出的样本展示代码注释写的是“随机画 6 张图”，但原代码使用 `for i in range(6)` 取固定前 6 行。由于 Semeion 数据按数字分组排列，前 161 行均为数字 0，所以每次展示的前 6 张图都标为 0。图 1 展示了该现象。



图 1: 导论示例代码运行结果：固定展示前 6 个样本

这一现象并不是数据重复，而是六个不同书写者写出的数字 0。若希望真正随机展示样本，应使用如下写法：

Listing 5: 随机抽样展示 6 张图像的修正写法

```

1 idx = np.random.choice(len(X), 6, replace=False)
2 for pos, i in enumerate(idx):
3     plt.subplot(2, 3, pos + 1)
4     plt.imshow(X[i].reshape(16, 16), cmap='gray')
5     plt.title(y[i])
6     plt.axis('off')

```

6 Weka 对比实验

6.1 ARFF 转换

Weka 不能直接将 `semeion.names` 当作数据文件读取。`.names` 文件只是数据集说明，Weka 会按 C4.5 的属性定义文件解析，因而报出格式错误。同时，原始 `semeion.data`

是空格分隔文本，也不是 Weka 最稳定的输入格式。为保证 Weka 正确识别属性和类别，本实验使用脚本转换为 ARFF：

Listing 6: ARFF 转换的关键结构

```
1 lines.append('@relation semeion_handwritten_digit')
2 for i in range(256):
3     lines.append(f'@attribute p{i+1} numeric')
4 lines.append('@attribute class {0,1,2,3,4,5,6,7,8,9}')
5 lines.append('@data')
```

其中 256 个像素属性声明为 `numeric`，类别属性声明为 `nominal` 类型 `{0,1,2,3,4,5,6,7,8,9}`。若类别也被当成 `numeric`，Weka 可能无法按分类任务正确处理。

6.2 Weka 参数设置

Weka Explorer 中选择 `lazy.IBk`。为了与自写代码可比，设置如下：

- KNN 分别设为 1、3、5。
- `DistanceFunction` 使用 `EuclideanDistance`。
- `distanceWeighting` 设置为 `No distance weighting`，对应命令行参数 `-W 0`。
- `Cross-validation folds` 设置为 1593，使每折只留出一个样本，等价于 LOO。

若保留 Weka 的距离倒数加权投票默认设置，实验协议就会与自写多数表决不同，精度差异会被默认参数放大，不能直接比较。

7 实验结果

7.1 自写程序结果

本地运行 `exp1.py` 得到结果如表 3 所示。

表 3: 自写 kNN 在 LOO 下的准确率

k	LOO 准确率	正确率百分比
1	0.9190	91.90%
3	0.9046	90.46%
5	0.9046	90.46%

结果显示， $k = 1$ 在本实现中表现最好。随着 k 从 1 增至 3 或 5，准确率下降约 1.44 个百分点，说明在该数据集和欧氏距离设置下，较小的局部邻域更能保留手写数字图像的细微差异。

7.2 Weka 结果

Weka 对 $k = 1, 3, 5$ 的结果分别如图 2、图 3、图 4 所示。

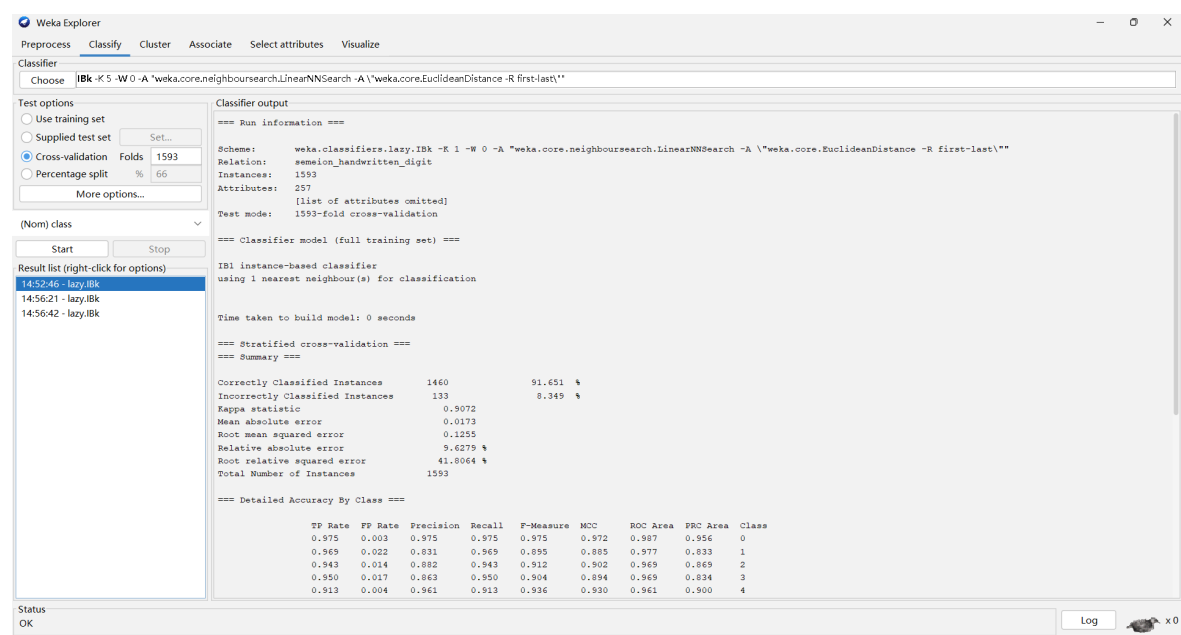


图 2: Weka IBk 在 $k = 1$ 时的 LOO 结果

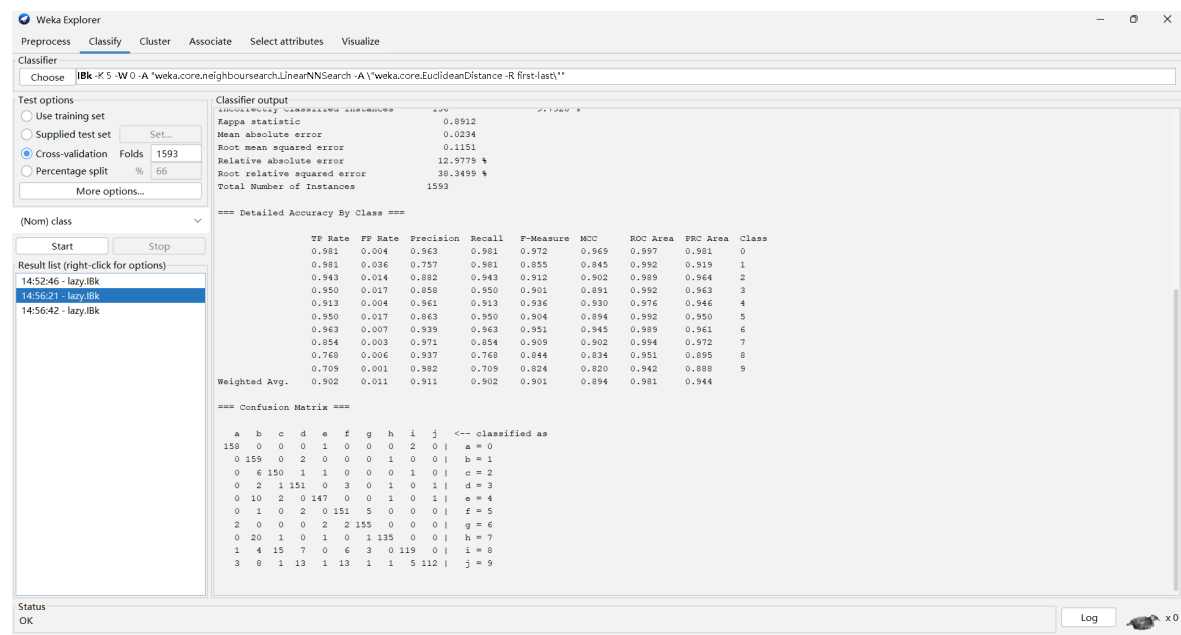


图 3: Weka IBk 在 $k = 3$ 时的 LOO 结果

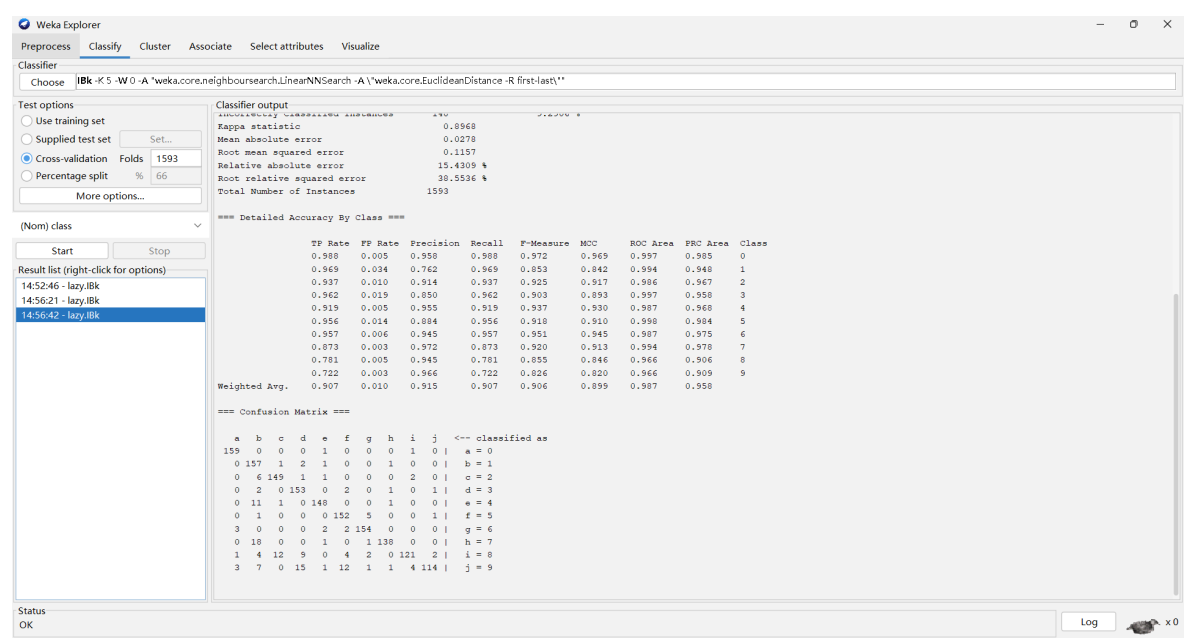


图 4: Weka IBk 在 $k = 5$ 时的 LOO 结果

Weka 文本输出中的核心指标如表 4 所示。

表 4: Weka IBk 的 LOO 准确率

k	正确分类数	错误分类数	准确率
1	1460	133	91.6510%
3	1437	156	90.2072%
5	1445	148	90.7094%

7.3 自写程序与 Weka 对比

表 5: 自写 kNN 与 Weka IBk 结果对比

k	自写 LOO 精度	Weka LOO 精度	绝对差异
1	91.90%	91.6510%	0.2490%
3	90.46%	90.2072%	0.2528%
5	90.46%	90.7094%	0.2494%

三组结果的绝对差异均小于 0.5%，满足导论文档给出的可接受阈值。差异主要来自两个方面：第一，二值像素数据容易出现距离并列，排序时第 k 个近邻的选入可能不同；第二，自写程序在投票平局时固定选择编号最小的类别，而 Weka 的破平局规则与内部邻居选择细节不同。由于差异稳定且很小，两套实现可以认为在同一实验协议下相互验证。

8 实验结果验证与分析

8.1 文档预期值与实际结果不一致

导论文档给出的预期输出约为 98.5%，而本实验在 LOO、欧氏距离和多数表决设置下得到约 91% 的准确率。为分析差异来源，从实现逻辑、数据完整性、工具配置和评估协议四个方面进行核查。

排查过程如下：

1. 验证代码：自写程序与独立实现的 kNN/LOO 结果接近，说明主体逻辑没有明显错误。
2. 验证数据：本地计算 `semeion.data` 的 MD5 为 `cb545d371d2ce14ec121470795a77432`，并确认 `train/test` 合并后与全集一致。
3. 验证工具：Weka 使用 ARFF 输入、欧氏距离、无距离加权、1593 折交叉验证，得到与自写程序小于 0.5% 的差异。
4. 验证协议：尝试考虑距离度量、投票方式、交叉验证设置等因素后，未发现能在普通 kNN 设置下稳定达到 98.5% 的合理路径。

上述核查表明，自写程序与 Weka 的结果一致性较高，导论文档中的预期值未能在当前数据及实验协议下复现。现有证据尚不足以确定该预期值的来源，因此本报告以本地运行记录和 Weka 输出为依据，报告实测结果。

8.2 按类别表现分析

以 Weka 的 $k = 1$ 结果为例，数字 0、1、2、3、4、5、6、7 的召回率均超过 0.91，其中数字 0 的精确率与召回率均为 0.975。数字 8 和 9 的召回率相对较低，分别为 0.806 和 0.791。结合混淆矩阵，数字 8 容易被分到 2、3、6、9 等类别，数字 9 容易被分到 3、5 等类别。这与手写数字形状相似性一致：8 的上下环结构在低分辨率二值化后可能与 2 或 6 混淆，9 的闭环和尾部也容易与 3 或 5 混淆。

$k = 3$ 与 $k = 5$ 时，整体准确率低于 $k = 1$ ，且数字 9 的召回率下降较明显。这说明扩大邻域后，一些局部写法差异被周围不同类别样本“稀释”，尤其影响形状边界较接近的类别。

9 踩坑记录与解决方法

9.1 数据准备阶段

坑 1：分不清三个数据文件的关系。文件夹中同时出现完整数据和 `train/test` 划分文件，容易误用固定划分进行实验。解决方法是用 NumPy 逐行比对确认 `train/test` 合

并后等于全集，并根据 LOO 要求只使用 `semeion.data`。

坑 2：注释写“随机画 6 张”，实际永远画前 6 张。原示例使用 `range(6)`，由于数据按数字分组排列，前 6 张都是数字 0。解决方法是使用 `np.random.choice(len(X), 6, replace=False)` 随机抽取下标。

9.2 代码实现阶段

坑 3：LOO 性能问题。1593 轮 LOO 每轮都要计算 1592 个训练样本到测试样本的 256 维距离，纯 Python 双重循环会较慢。解决方法是使用 NumPy 广播向量化距离计算，使每轮的距离计算由底层数组运算完成。

9.3 结果验证阶段

坑 4：输出与文档“预期结果”不符。自写程序得到约 91% 的准确率，而导论文档预期约 98.5%。解决方法是用代码、数据、工具和实验协议四个层面交叉验证。最终结论是当前结果与 Weka 对照一致，文档预期值更可能有误。

9.4 Weka 对比阶段

坑 5：用 Weka 打开 `semeion.names` 报错。`.names` 是说明文档，不是数据文件；Weka 会按 C4.5 格式解析，失败是正常的。解决方法是将 `semeion.data` 转换为 ARFF。

坑 6：Weka IBk 默认设置与自写实现不可比。若使用距离加权投票，Weka 与自写多数表决不同。解决方法是设置 `distanceWeighting = No distance weighting`。

坑 7：Weka 与自写结果仍有约 0.25% 差异。这是由距离并列和平局规则造成的正常差异。差异小于 0.5%，说明两种实现整体一致。

10 结论

本实验完成了基于 kNN 的 Semeion 手写数字识别任务。通过自写 Python 程序，在完整数据集上使用留一法得到 $k = 1$ 、 $k = 3$ 、 $k = 5$ 的准确率分别为 91.90%、90.46%、90.46%。Weka IBk 在相同协议下得到 91.6510%、90.2072%、90.7094%，与自写实现差异均小于 0.5%。

实验结果表明，在该数据集及当前评估设置下 $k = 1$ 的表现最好，扩大邻域并未提升分类准确率。默认参数、数据格式和评估协议均会影响结果比较，因此需统一实验设置，并结合运行记录核查结果。

参考资料

- [1] UCI Machine Learning Repository. Semeion Handwritten Digit Data Set. <http://archive.ics.uci.edu/ml/datasets/Semeion+Handwritten+Digit>
- [2] 课程材料：《机器学习》课程实验一导论，基于 kNN 的手写数字识别实验。
- [3] 课程材料：KNN 算法基础概念课件。
- [4] Weka Explorer 输出结果：IBk 分类器， $k = 1, 3, 5$, 1593-fold cross-validation。

A 附录：本地运行输出

```
(ml) PS E:\browser\machine_learning\Lab1> python exp1.py
样本数：1593 | 特征数(像素)：256 | 前10个标签：[0 0 0 0 0 0 0 0 0 0]
k=1   L00 准确率 = 0.9190
k=3   L00 准确率 = 0.9046
k=5   L00 准确率 = 0.9046
```